Text-to-SQL Models Make Choices Without Saying So [Experiment, Analysis & Benchmark]

Yichen Xu
University of California, Berkeley
Berkeley, CA, USA
yichen_xu@berkeley.edu

Shipei Lin
School of Data Science
The Chinese University of Hong Kong, Shenzhen
Shenzhen, China
225045017@link.cuhk.edu.cn

Chenhao Ma
School of Data Science
The Chinese University of Hong Kong, Shenzhen
Shenzhen, China
machenhao@cuhk.edu.cn

ABSTRACT

A text-to-SQL answer can run, return a plausible number and still compute something other than what the analyst meant. This happens when a request leaves a calculation choice open, such as whether to count rows or entities, and the model settles the choice without saying so. Benchmarks cannot detect this, because they score one reference reading even where the question leaves the choice open. This paper introduces SpecBridge, a framework that tests what a model does with such a choice by changing only that choice. A request set holds three requests on the same data, one for each of two readings and one that leaves the choice open, and execution scores every answer. SpecBridge builds request sets on real databases from choices mined from benchmark failures. On 204 hand-validated request sets, four models usually compute both readings when each is asked for, yet for the open request they return both readings in only 16–58% of those sets, and they seldom say that a choice existed. When they settle a choice silently, they take the reading that benchmark references take by convention, and on BIRD over two in five of the answers scored wrong on such questions return the other reading. Two design rules make answers more reliable. Showing an alternative only when its result differs is 19 points more precise than rewriting the request once per reading, and a clarification is best carried by adding one clause rather than by rewriting the request.

KEYWORDS

text-to-SQL, database interfaces, evaluation, controlled tests, ambiguity, language models

PVLDB Reference Format:
Yichen Xu, Shipei Lin, and Chenhao Ma. Text-to-SQL Models Make Choices Without Saying So [Experiment, Analysis & Benchmark]. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at https://github.com/Xyc-arch/specbridge-artifact.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Analysts increasingly ask for numbers in natural language and get them from a text-to-SQL system, often without reading the SQL. For these users a query that fails is safe, because the error is visible. The dangerous answer is a query that runs and returns a plausible number, computed differently than the analyst meant. Outside SQL, such small calculation choices have had large consequences. A finding that high public debt slows growth depended on averaging within each country before averaging across countries [6], and a health-care algorithm that measured need by cost cut the Black patients chosen for extra care by more than half [17]. Hospital mortality ratings depend on which discharges count [25], and twenty-nine teams given the same data on referees and skin tone reached different conclusions [26]. A text-to-SQL system makes such choices on every request, and its user sees only the result.

Take "the average rental price per day" for each film rating (Figure 1). It can be computed two ways, which we call readings. Reading A averages each film's daily price, and reading B divides the average price by the average rental length. For two films priced at \$6 for two days and \$6 for six days, the first reading gives \$2.00 per day and the second \$1.50. Both are simple SQL, and on the `sakila_1` database the two readings differ for every rating. For the request as worded, which leaves this choice open, all four models we test, Claude Haiku 4.5, Claude Sonnet 5.5, GPT-6 Sol and GPT-6 Luna (Haiku, Sonnet, GPT-Sol and GPT-Luna below), return A; only Sonnet also says that there is a choice and gives B, the ratio of two averages.

A reliable text-to-SQL system must compute the reading that a specific request asks for, report a choice that a request leaves open together with the other reading, and stay quiet about a choice that the wording already settles. Benchmarks such as Spider and BIRD measure only the first. They score each question against one reference reading, even where the question leaves the choice open, so a model that guesses the authors' reading scores as well as one that reports the choice. Ambiguity benchmarks [1, 22] score whether a model's outputs contain every reading of an ambiguous question. They cannot separate a model that cannot compute a reading from one that computes it but stays silent, and they do not penalize raising a settled choice (Table 1).

Our key idea is to change only the choice. For each test we write three requests on the same database: a specific request for reading A, a specific request for reading B, and the open request, which leaves the choice open. With the two reference queries they form a request set. Because the three requests differ only in the choice, each failure has one cause. The model cannot compute a reading,

Request on `sakila_1`: "For each film rating, what is the average rental price per day?"

read as A | read as B

Reading A: mean of daily prices

```
SELECT rating,
AVG(rental_rate /
  rental_duration)
FROM film GROUP BY rating
```

result: G 0.668, PG 0.642, R 0.662

Reading B: ratio of two averages

```
SELECT rating,
AVG(rental_rate) /
AVG(rental_duration)
FROM film GROUP BY rating
```

result: G 0.597, PG 0.600, R 0.616

Request set (shortened): wording, and the answers counted correct

specific, A	"the average of each film's daily price"	A
specific, B	"average price divided by average duration"	B
open	"average rental price per day"	A or B, choice named

Four models' responses to the open request

	Haiku	Sonnet	GPT-Sol	GPT-Luna
returns A	✓	✓	✓	✓
names the choice	✗	✓	✗	✓
also returns B	✗	✓	✗	✗

Figure 1: One request, two correct readings (shaded: the SQL that differs), and four models' responses. Each specific request asks for one reading; the open request keeps the original wording.

Table 1: SpecBridge and the two closest benchmarks of ambiguous text-to-SQL. Schema: synonym or split columns and tables; wording: scope, attachment, vagueness; calculation: ties, counting units, missing values, aggregation level. Mined: in our library of choices from benchmark failures, but not built into request sets.

	AmbiQT [1]	AMBROSIA [22]	SpecBridge
Choices in the schema	✓	×	mined
Choices in the wording	×	✓	mined
Choices in the calculation	×	×	✓
Benchmark databases, unmodified	×	×	✓
Measures reference conventions	×	×	✓
One request per reading	×	✓	✓
Separates computing from reporting	×	×	✓
Checks the stated reading	×	×	✓
Penalizes raising a settled choice	×	×	✓

computes one but picks the other, or computes both but does not tell the user. For the open request, the model states its assumptions and may give SQL for other readings; offering the other reading for a specific request is a false alarm, as the wording settles it.

Building request sets at scale is hard for three reasons. First, a test must control what each query computes and describe it exactly, on databases that nobody has annotated. Second, the choices must be ones that matter in real requests, not invented ambiguities. Third, the three requests must differ only in the choice, yet a rewritten request easily changes other requirements. SpecBridge addresses each in turn. It generates SQL from a catalogue of column meanings under type rules, and code describes each query exactly in words (Section 2.2). It mines choices from BIRD and Spider questions that models fail as worded but answer correctly from an exact description of the reference SQL, and keeps only edits that change the answer on the data (Section 2.3). It writes the three requests from one list of shared requirements, and code, two reviewer models and an author check every set (Sections 2.4 and 4.2). Execution, not a model, decides every accuracy score.

Finding 1: Models make choices without saying so (Section 5). On 204 accepted request sets, the models answer both specific requests correctly in most sets, but on the open request they return both readings in only 16–58% of those sets. Most missed readings are ones the same model computes correctly when asked for them alone, so the gap lies in reporting, not in computing. When a model settles a choice silently, it takes the reading that benchmark references take by convention, the reading benchmarks reward.

Finding 2: Benchmarks cannot see the gap (Section 6). When the wording of a BIRD or Spider question does not settle a choice, the reference SQL still takes one reading, by a convention of its author group. BIRD and Spider's own authors return one row when several tie and count joined rows, while the questions Spider imported from older datasets reverse one of these conventions each. The choice is not cosmetic. On BIRD, nearly one in four questions that leave a choice open changes its answer under the other reading, and over two in five answers BIRD marks wrong give that reading.

Finding 3: Two design rules make answers more reliable (Section 7). Most requests are clear, so a system must not add wrong alternatives to them. Answering each reading separately and showing an alternative only when its result differs is 19 points more precise on AMBROSIA than disambiguate-then-parse (DtP) [23], which rewrites the request once per reading, at a third of its calls. When the user settles a choice, the system should keep the request and add one clause, because a rewritten request loses other requirements, such as an output column or a join.

In summary, we make three contributions: (1) request sets, controlled tests that change one calculation choice and are scored by execution, with a generator that builds them on real databases and a library of choices mined from benchmark failures (Sections 2–4); (2) three findings on four models across our request sets, AMBROSIA and BIRD (Sections 5–6); and (3) two design rules for reliable answers, each tested against the closest alternative (Section 7). We release the generator, the 307 request sets with the author's verdicts, all model responses, and a package that reproduces every number in this paper without new model calls; a supplement shows how common each choice type is, with examples, and gives the generator's details.

2 SPECBRIDGE OVERVIEW

SpecBridge turns a database into request sets in two lanes that meet in one step (Figure 2). The first lane generates queries whose computation is known exactly. The second lane mines, from benchmark failures, the choices that real requests leave open. The last step applies one such choice to a generated query and writes the three requests. We first follow two request sets through this process (Section 2.1) and then describe the steps of generating queries (Section 2.2), mining choices (Section 2.3), and building request sets (Section 2.4).

2.1 Two examples

A request set fixes a database and every requirement of a query but one choice. The film request of Figure 1 asks for each rating's average replacement cost and daily rental price, and compares that

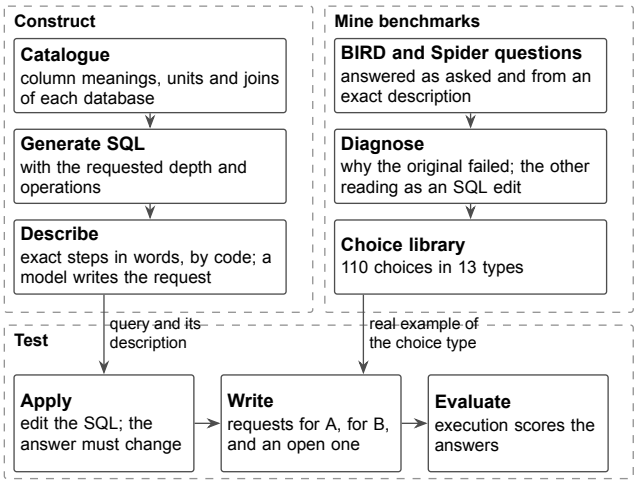


Figure 2: The SpecBridge pipeline.

daily price with the average across ratings. Only the daily price is open:

$$A = \mathrm{AVG}(\mathrm{price}/\mathrm{days}), \qquad B = \frac{\mathrm{AVG}(\mathrm{price})}{\mathrm{AVG}(\mathrm{days})}. \tag{1}$$

A averages each film's daily price; B divides two averages and so gives more weight to longer rentals. The specific request for A asks for the average of per-film daily prices, the one for B asks for the average price divided by the average duration, and the open request asks for the "average rental price per day." The rest of the wording is shared, and code checks that the two queries return different results on the database; a choice that does not change the answer is not tested there. The second example (Figure 3) is a three-step query on the `insider` database. It averages short-interest measurements for each transaction, compares each transaction with a baseline for its order type, and returns the ten largest ratios. The choice lies in the baseline. Reading A averages the transaction means, so each transaction counts once, while B averages the underlying snapshots, so a transaction with more snapshots weighs more. The open request asks for "the average short interest for its order type mix," and all ten output rows change between the readings.

2.2 Generating queries

Generation starts from the database, not from a benchmark, so it works on databases nobody has annotated. One model review of the schema and data first builds a catalogue of column meanings, units and table relationships; after that, generation calls no model. A query starts from counts, sums or averages per entity, per group or overall. Later steps can compare a value with a broader average, take a share of a total, combine compatible quantities, aggregate again, or keep the top entities per group. We control two properties of each query, its arithmetic depth (the nesting of arithmetic expressions) and its step depth (the number of dependent query stages). Type rules decide which quantities can be combined or summed, and data checks reject empty answers, constant or redundant quantities, non-positive denominators and too-small groups. To reach a requested depth without trial and error, the generator looks ahead and drops a

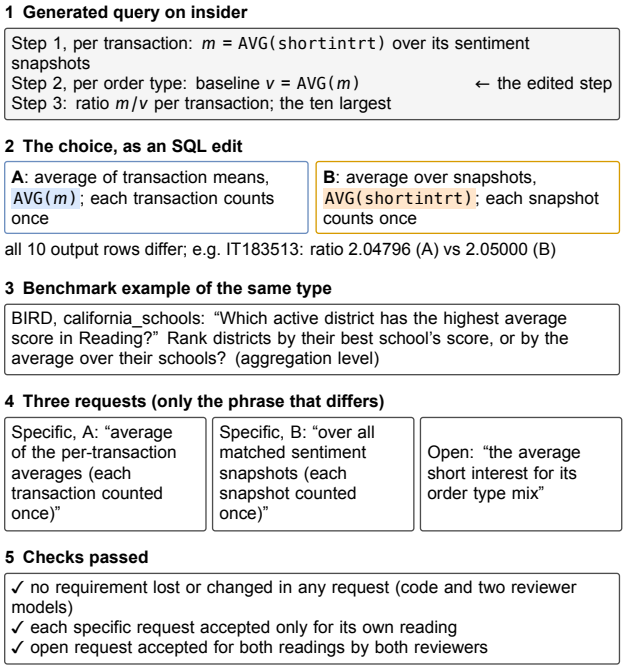


Figure 3: A request set on `insider`, from the generated query to its checks. Shaded: the SQL that differs.

partial query as soon as no completion of it can reach the requested depths and operations, so attempts are not spent on dead ends.

Code then describes each query exactly in words. The text covers source rows, joins, unmatched rows, filters, groups, formulas, missing values, ordering, limits, ties and SQLite conventions, and the same SQL always gives the same text; we call it the exact description. Because the exact description states every requirement, a model that fails from it cannot compute the query, while a model that succeeds from it but fails from a natural request was misled by the wording. Section 2.3 builds on this contrast. A writer model turns the exact description into one natural request, using the column meanings. A second model checks the request for missing, changed or added requirements, and code checks how stored values are written. A failing request gets one revision and is dropped if it fails again.

2.3 Mining choices from benchmark failures

The choices come from real benchmark failures. For each BIRD and Spider question, we compare a model's answer to the original question (with BIRD's evidence) with its answer to the exact description of the reference SQL. When only the description is answered correctly, the wording caused the failure, not the model's SQL, and we classify the reason as output columns, a different column or value, an unstated requirement, numerical precision, or an open choice. For an open choice, the other reading is written as an edit of the reference SQL, and both queries are executed. Two reviewer models judge each reading independently, and disagreements are kept. The library keeps choices of meaning, both open ones, where the wording fits both readings, and settled ones, where it selects one. For "What is the total number of molecules with double bonded

Choice type	Example open phrase	One reading	Other reading
Aggregation level rows or group averages?	“average standings points at races held there”	pool all records	average each race first
Strict cutoff strict or inclusive?	“using 0.149 as the cutoff”	greater than 0.149	at least 0.149
Ties one tied row, or all?	“the top 10 telescopes by share of signals”	exactly ten rows	every telescope tied with the tenth
Counting unit rows or entities?	“how many manager award winners”	distinct managers	award records
Missing values keep or skip missing?	“group members by their major”	no major is its own group	members without a major left out
Referent column which column is meant?	“schools in Riverside”	the district field	the county or the city field
Seven other types e.g. conditions on one record?			

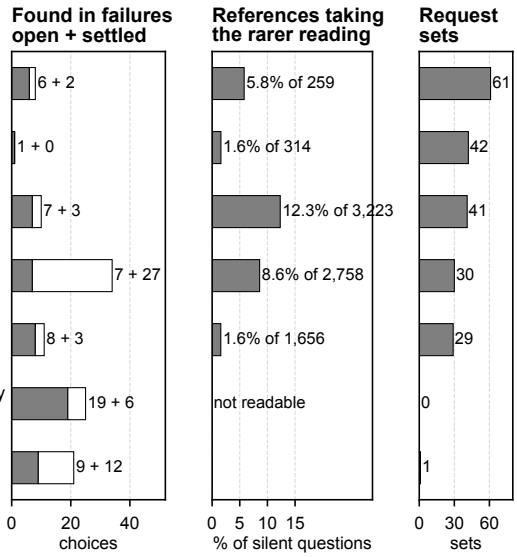


Figure 4: Choice types: an example open phrase with its two readings, choices found in benchmark failures (gray: open; white: settled), silent BIRD and Spider references that take the rarer reading, and accepted request sets.

oxygen?”, both reviewers accept only the reading in which the oxygen atom is part of the double bond, although the reference SQL counts every molecule that contains an oxygen atom and a double bond. Figure 4 groups the library by choice type. The largest group of open choices concerns which column a phrase refers to, and half of the settled ones concern the counting unit.

The same choices appear in benchmark references, and the references do not agree on them. We call a question silent on a choice when neither its wording nor BIRD’s evidence contains a word that settles it, such as “all” for ties or “distinct” for counts. For 12 of the 13 choice types, code reads which reading a reference takes, and Figure 4 counts the silent BIRD and Spider questions and the share whose reference takes the rarer reading. Every case of the rarer reading was reviewed and only confirmed cases are counted, so these shares are conservative. For example, 12.3% of silent tie questions return every tied row, and 8.6% of silent counts over a join count distinct entities. For these choices, a benchmark that scores one reading treats a convention as the only correct answer.

2.4 Building request sets

A request set needs a choice that matters on the data, so code first reports where a generated task’s columns have missing values, repeated join matches, distinct values or ties. A writer model proposes one choice from the library and an exact SQL edit for it, and code checks that the edit applies, runs, changes the answer and can still be described exactly; a failed proposal gets one repair. The writer then writes the request set from a plan that lists the shared requirements and the one differing choice. It writes the request for A, edits it for B, and writes the open request. A library example of the same type may guide the wording, but never supplies the SQL. Two reviewer models check that the requests keep their requirements and judge each of the three requests against each reading. A set is kept only when both reviewers accept each specific request for its own reading alone and at least one accepts the open request for both readings. Five library types concern a filter, such as whether a cutoff is strict. Generated queries seldom filter, so for these types the generator adds one condition chosen from the data, which keeps 20–80% of the rows. Figure 4 gives an example of each type. The pipeline keeps 307 request sets on 12 databases, and 204 of them pass an author’s review and a check against real references (Section 4.2).

3 MEASURES

We score what a model computes by execution and what it says by a model judge. A specific request selects one reading, and the open request fits both. A choice is settled when the wording selects one reading. We execute every returned query and compare its result with the reference result, with numerical tolerance and with rows tied at a cut-off treated as interchangeable; an execution error scores zero, as in Spider and BIRD scoring. For benchmark questions we also report a score that allows extra or reordered output columns.

For the open request, the model returns a main query and up to two alternatives. The model returns both readings when these queries include one with the result of A and one with the result of B. We report this share over all sets and over the sets in which the same model answers both specific requests correctly, where a miss cannot be a failure to compute. Written as a formula, with S_i the queries returned for the open request of set i, $e(p, \hat{p}) = 1$ when query $\hat{p}$ returns the result of reference p, and $B_i = 1$ when both specific requests are answered correctly,

$$\text{Both} = \frac{\sum_i B_i \prod_{j \in \{A,B\}} \max_{\hat{p} \in S_i} e(p_j, \hat{p})}{\sum_i B_i}. \tag{2}$$

Table 2: Four recorded responses and how the measures score them.

Request, model	Main SQL	Alternative SQL	Names	Both	False alarm
Film, open, GPT-Sol	A: mean of daily prices	none	×	×	
Film, open, GPT-Luna	A: mean of daily prices	none	✓	×	
Awards, open, Sonnet	A: `COUNT(*)`	B: `COUNT(DISTINCT player_id)`	✓	✓	
Fees, specific, Haiku	A, as asked	B: mean of channel means			✓

Table 3: The data and where each is used.

Data	Contents	Sections
Request sets	Spider: baseball, Sakila, customers-and-invoices, Chinook. BIRD: financial, student club, thrombosis prediction, Formula 1. LiveSQLBench: archeology, insider, crypto, alien	5, 7
BIRD development	481 questions on five databases to mine the library; 100 held out	2.3, 6
Spider development	150 questions to mine the library; 50 held out	2.3, 6
BIRD and Spider	references of all questions silent on a choice	2.3, 6.1, 6.2
AMBROSIA test	ambiguous and unambiguous questions	5, 7

A missed reading is computable when the same model answers the specific request for it correctly. A model names the choice when its explanation says that the choice is open, whether or not it gives SQL for the other reading. A judge model, which does not know which model answered, decides whether a response names the choice and whether its stated reading matches its SQL. For a specific request, offering the other reading, or calling the choice open, is a false alarm. The measures check one another. Returning both readings alone would reward a model that adds every alternative it can think of, and the false alarms charge for that; naming alone would reward a warning without an answer, and returning both readings charges for the missing query. Table 2 scores four recorded responses.

On AMBROSIA, each ambiguous question comes with its readings, and a reading counts as found when some returned query gives its result. Recall is the share of readings found, and AllFound is the share of questions with every reading found [22]. Precision on an unambiguous question is the share of returned queries that give its one correct result, so extra wrong readings lower it.

4 EXPERIMENTAL SETUP AND TEST CHECKS

Sections 5–7 ask whether models report the choices they make, whether benchmarks can see if they do, and how a system should disclose a choice and carry a clarification. This section describes the setup (Section 4.1) and checks that the tests change only the choice (Section 4.2).

4.1 Experiment settings

Data. Request sets use 12 databases, four each from Spider, BIRD development and LiveSQLBench[1]. The generated tasks request arithmetic depth 1–2 and step depth 3–6, and each database contributes 12 to 24 accepted request sets. The library is mined from BIRD and Spider development questions, and a part of each benchmark is held out until the final evaluation (Section 6), so no held-out question shaped the library. Table 3 lists the data and where each is used.

Models. We evaluate Haiku, Sonnet, GPT-Sol and GPT-Luna (`claude-haiku-4-5`, `claude-sonnet-5-5`, `gpt-6-sol`, `gpt-6-luna`). Claude Opus 5.5 writes and reviews the requests and judges the explanations, with Sonnet as the second reviewer; it is not evaluated. Models see the schema and documentation and have no file or tool access. By default a model answers in one call, with a prompt that asks for SQL, for a brief statement of "any consequential assumption that the request and the supplied context do not settle," and for SQL of any other supported reading, without inventing alternatives (the explain prompt). To test whether a model can compute a reading at all, we give it its best chance and let it explore. It may first run up to six rounds of five queries on the database, seeing twenty rows of each result. Exploration changes little; GPT-Sol answers 59 of 100 original BIRD questions correctly without it and 61 with it, so it does not explain the results below.

Statistics and baselines. Intervals resample tasks 2,000 times, keeping pairs together; proportions use Wilson intervals; paired comparisons use exact McNemar tests. For the disclosure method (Section 7), we compare with the AMBROSIA authors' prompt, the same prompt sampled three times at about the method's cost, and disambiguate-then-parse [23].

4.2 Checks on the tests

The findings rest on tests that change only the choice, so we check the tests first. Figure 5 follows the sets through construction. Its largest cuts come from keeping one choice per task and from the author's review. Code confirms that in each of the 307 sets that pass all checks, the two queries differ in one step and the two specific requests differ only in the choice phrase. An author then read every set without seeing model results and kept it only if a reasonable analyst could mean either reading. Most rejected sets have an open request whose wording already favours one reading. A choice can also look open and still be settled by convention. Every date-range request says "between," which SQL's `BETWEEN` includes, and 254 of the 255 BIRD and Spider references whose question says "between" with no other cue include both ends, so the author rejects all 17 date-range sets. Last, a set must test a choice on which real references differ, so we drop the types whose other reading fewer than 1% of silent references take (Figure 4). The author keeps 232 sets, and this rule leaves 204, which test aggregation level, cutoffs, ties, counting units, missing values and one negation. Every result below uses these 204 sets.

The generator itself is reliable. Its rules find usable queries on 152 of 190 databases; the others lack the repeated rows the rules need. On the 12 evaluation databases it delivers 1,198 of 1,200 requested queries, all at the requested depths, and each of its rules is needed (Figure 6); without the look-ahead, almost no attempt reaches its target. Natural requests also lose little against exact descriptions (Figure 7). Sonnet and GPT-Sol lose about two points, and Haiku and GPT-Luna do better with natural wording. Because Sonnet writes every natural request and is also evaluated, GPT-Sol wrote a second version of 200 random tasks; measured against the two

[1] https://github.com/bird-bench/livesqlbench

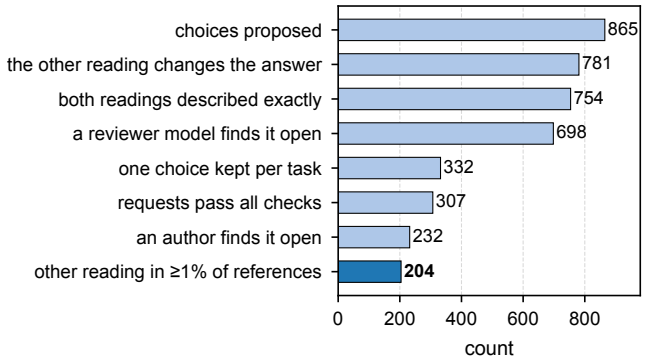


Figure 5: What each construction stage keeps, from proposed choices to accepted request sets, with and without a filter.

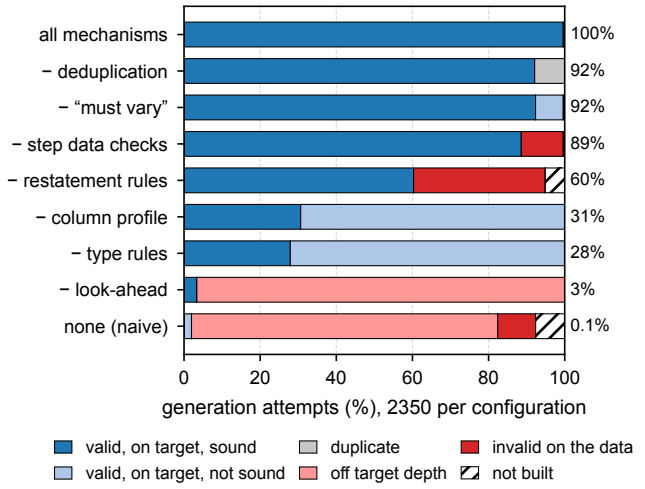


Figure 6: Generation outcomes with each construction rule removed (2,350 attempts). The labels give the share valid, on target and sound.

models that wrote neither, neither writer favours itself. Writing from a plan keeps the three requests aligned. On 60 fixed tasks, it keeps the shared requirements in 52 drafts, against 32 for direct generation ($p = 0.0002$), and passes all checks in 49 against 32.

5 MODELS MAKE CHOICES WITHOUT SAYING SO

A model that computes both readings when each is asked for usually returns only one of them for the open request, and seldom says that there was a choice. Asking for an explanation does not cause this gap. Against the same call asking only for SQL, correct answers to specific requests change by at most three of 204 per model, and every paired interval includes zero. Nor does it make models raise settled choices. The judge finds at most two of 204 responses to specific requests per model that call the choice open. The explanation costs time. The median request takes 7.5–50.0 seconds per model with it, against 4.3–17.9 seconds for SQL alone.

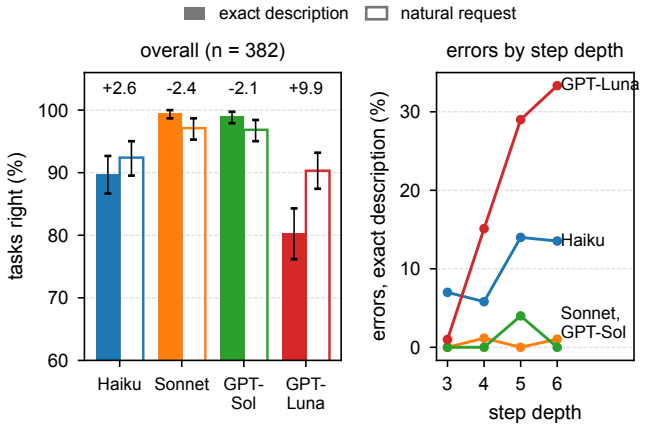


Figure 7: Exact description versus natural request on 382 generated tasks. Left: accuracy; numbers give the difference in points. Right: errors from the exact description by step depth.

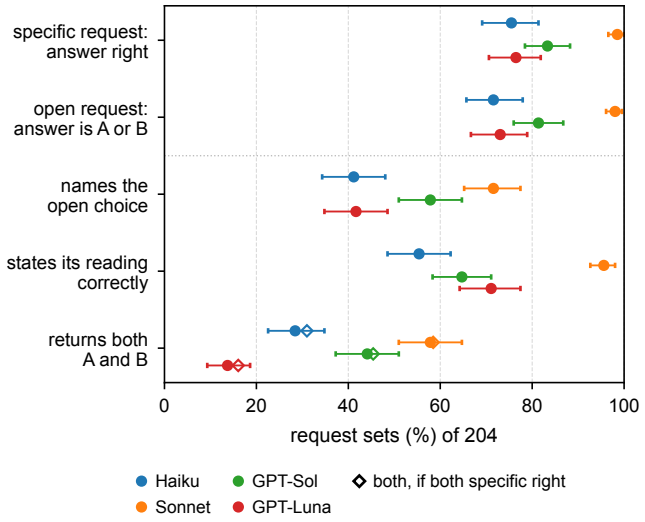


Figure 8: Answering and communicating on 204 request sets (95% intervals).

Models return one reading where two are correct. Returning both readings is much rarer than returning one correct answer (Figure 8). Even among the sets in which a model answers both specific requests correctly, it returns both readings for the open request in only 58% for Sonnet, 45% for GPT-Sol, 31% for Haiku and 16% for GPT-Luna. What the models state is accurate. A stated reading matches the model's SQL in 99–100% of responses. The models describe one calculation correctly without saying that another, equally valid calculation was possible on the same data. The unreported reading matters. In 98 of the 204 sets, the two readings return different rows, and in the other 106 the same rows with different computed values, so the user gets a different answer in every set.

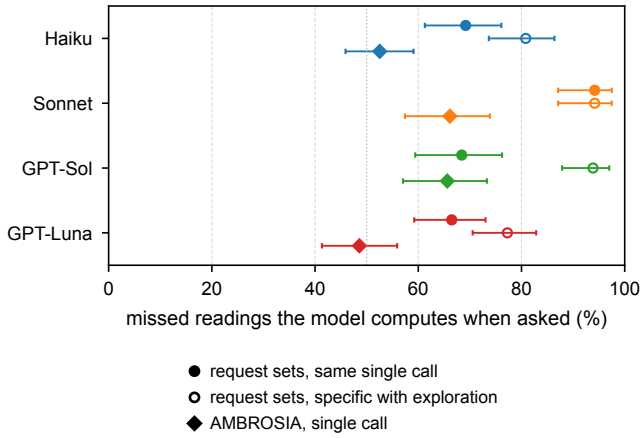


Figure 9: Missed readings that the same model computes when asked for them alone, on request sets with one call or after exploring the database, and on AMBROSIA (Wilson 95% intervals).

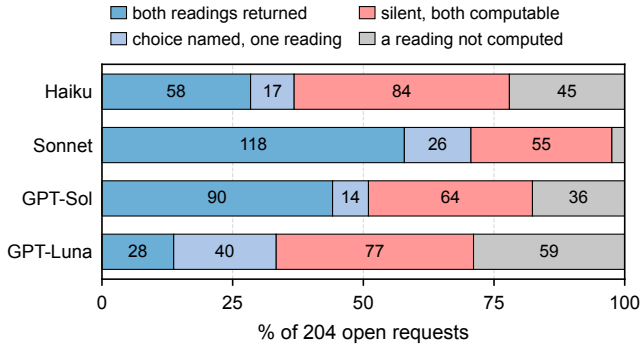


Figure 10: What each model does with the 204 open requests (one call with explanation). Computable: the same call answers both specific requests correctly.

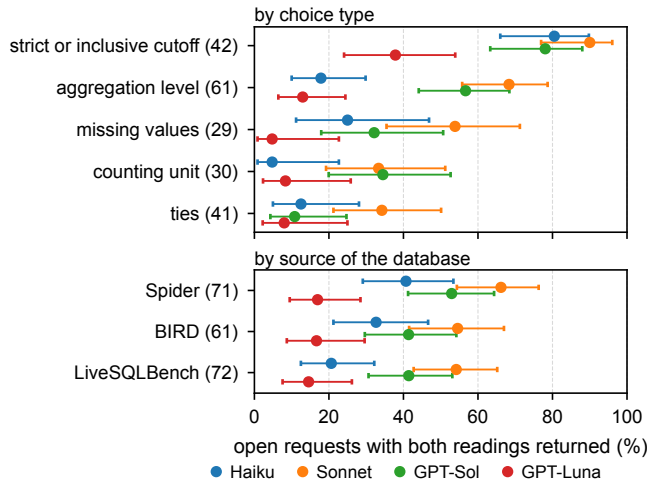


Figure 11: Open requests answered with both readings, by choice type and by source of the database, among the sets in which the same model answers both specific requests right (accepted sets in parentheses; Wilson 95% intervals).

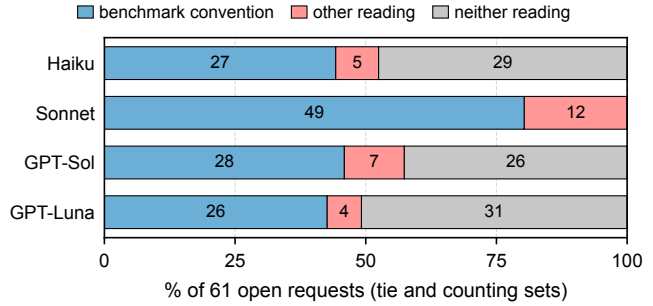


Figure 12: Main answer to the open request on 61 tie and counting sets: the reading BIRD and Spider references take by convention, the other reading, or neither.

Most misses are failures to report. Most readings a model misses on the open request are ones it can compute. The same model computes the missed reading correctly when asked for it alone in 66–94% of the missed sets (Figure 9). Figure 10 sorts every open request into four outcomes, namely both readings returned, the choice named but one reading given, one reading given silently although the model computes both, and a reading the model does not compute. For Haiku and GPT-Luna the most common outcome is the silent one, in which the model can compute both readings but gives one without naming the choice, and for Sonnet and GPT-Sol it is the second most common. A score that only checks whether both readings appear among the outputs, as top-k coverage [1] does, cannot tell these outcomes apart. Sampling does not surface the other reading either. Five SQL-only samples contain both readings in at most 16 of 55 sets for any model, while one explained response from Sonnet does so in 34. What is missing is a prompt to report the choice, not more samples of the same answer.

The gap is widest for choices inside a calculation. Sonnet, the best model, returns both readings for 68% of aggregation-level sets but for only a third of tie and counting-unit sets, and the other models do worse (Figure 11). These choices hide in the SQL, in a `LIMIT`, a `COUNT` over a join, or an average taken at one level, none of which the request mentions. Cutoffs are the exception, because most cutoff requests give a value "as the cutoff" with no comparison word, which invites both the strict and the inclusive reading. The gap holds on every source of databases, and no model returns both readings for more than 67% of the sets from any source.

Naming a choice is not the same as giving the other reading. A model that says a choice is open often stops there. Among the sets in which a model names the choice and can compute both readings in the same call, it still returns only one reading in 14–61% of them, and GPT-Luna does so most often. Such a response warns the user but leaves them to write the other query, which is a step the model could have taken itself, since it computes that query when asked.

Silent choices follow benchmark conventions. When a model settles a choice without saying so, it takes the reading that benchmark authors take. In 61 tie and counting sets, one reading is the convention of BIRD and Spider references (Section 6.1), namely a plain `LIMIT` rather than every row tied at the cut-off, and counting joined rows rather than distinct entities. The conventional reading is the intended one in 29 of these sets, so the design favours neither. Yet when a model's main answer is one of the two readings, it is the conventional one in 80–87% of cases for every model (Figure 12). The conventional reading is also the simpler SQL, so these data cannot separate habit from simplicity. Either way, models settle an open choice the way benchmark authors do, which is also the reading that a benchmark built by those authors rewards.

The same holds on AMBROSIA. The pattern is not an artifact of our tests. AMBROSIA pairs each ambiguous question with one unambiguous question per reading, and Saparina and Lapata [23] observe that models usually give one preferred reading. We split each missed reading into one the model cannot compute and one it computes but does not report. On 120 ambiguous test questions per model, the model computes 49–66% of the readings it misses when asked the unambiguous version (Figure 9). Even when a model answers every unambiguous version correctly, it returns all readings for the ambiguous question in only 7–28% of questions. On the 40 vagueness questions per model, which ask which column or value a phrase means, models compute 48–56% of the readings they miss, so the gap also covers the referent-column choices that our request sets leave out. On both benchmarks, models compute what is asked and stay quiet about settled choices, but leave open choices unreported, whatever the source of the ambiguity.

Examples. For the film request of Figure 1, GPT-Sol states reading A, which matches its SQL, but treats the choice as settled; GPT-Luna says the choice is open but gives no second query; Sonnet adds B: "per-day price defined as the rating's average rental_rate divided by its average rental_duration (ratio of averages)." Responses can also mislead in the other direction. For a specific request on fees, Haiku computes a mean of channel means, which matches the other reading and contradicts the request; for an open count of awards, Sonnet names the choice and gives both `COUNT(*)` and `COUNT(DISTINCT player_id)`.

6 BENCHMARKS CANNOT SEE THE GAP

BIRD and Spider score a model against one reading even where a question leaves the choice open, and which reading they take depends on who wrote the question. On held-out questions, the four models answer 49–74% of BIRD questions correctly as worded but 96–100% from the exact description of the reference SQL (Figure 13). Allowing extra or reordered output columns closes only part of this gap, and on Spider the remaining gap is smaller.

The exact description gives the reference's steps, columns and choices at once, so this gap bounds what the wording costs rather than measuring ambiguity alone. To see what it consists of, a reviewer model assigns one cause to each development question answered correctly from the description and wrongly as worded (Figure 14). Output columns, a phrase mapped to another column or value, unstated requirements and run-to-run variation cause most failures. Output columns are a matter of form, as the answer

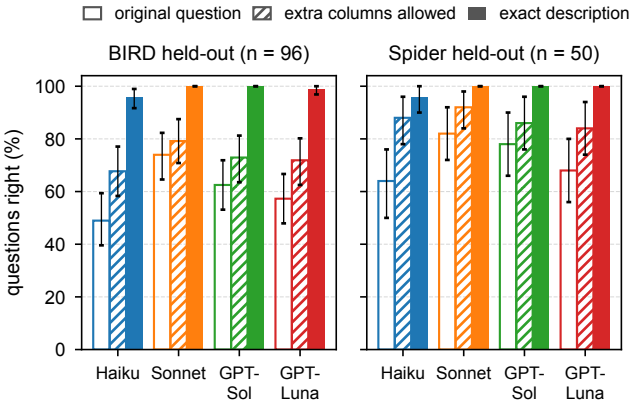


Figure 13: Held-out benchmark questions: original question versus the exact description of the reference SQL (95% intervals).

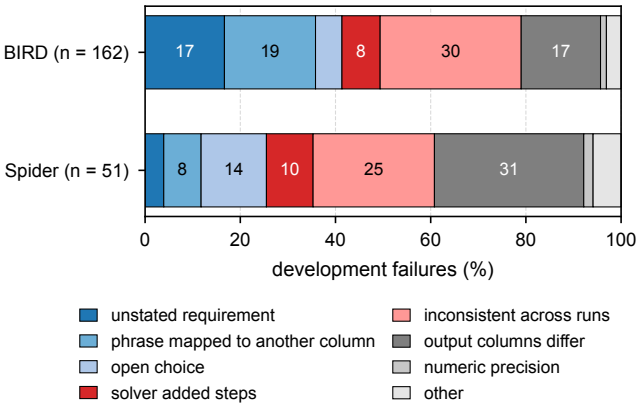


Figure 14: Why the original question failed, on development questions answered right from the exact description and wrong as worded.

holds the right values in a different shape, such as a name instead of an ID. Open choices cause 6% of BIRD and 14% of Spider failures. These open choices are the ones our library collects.

Other choices are about which column a word means. For "Name schools in Riverside…", filtering the district field returns six rows, the county field sixty and the city field seven. Code cannot tell from a reference which column the question's word could also mean, so request sets do not test this type; AMBROSIA's vagueness questions cover it.

6.1 References follow unstated conventions

When a question is silent on a choice, its reference still takes one reading, and the reading depends on the author group. We read seven such choices off reference SQL, among them ties at an extreme, counting over a join, join type, and whether "between" includes its bounds. On five of the seven, every author group takes the same reading in at least 91% of silent questions. On ties and counting, the groups disagree (Figure 15). Asked for an extreme

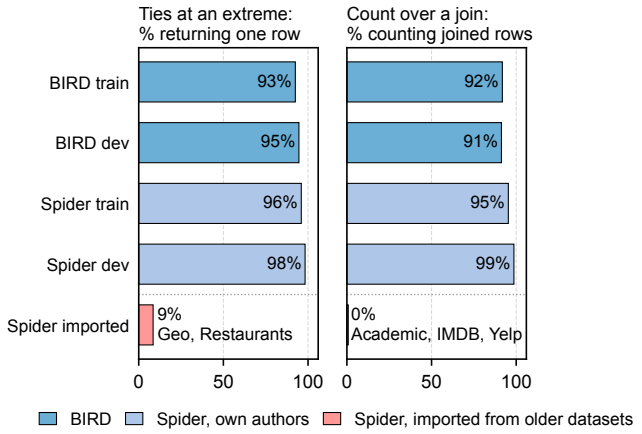


Figure 15: Share of silent questions whose reference takes the usual reading, by author group, for the two choices on which groups disagree.

Table 4: Six of the 18 BIRD questions on which all four models return the other reading, confirmed by an author; BIRD scores each of these answers wrong.

Question	BIRD reference	Other reading
"Who is the tallest superhero?"	1 hero	2 tied heroes
"… cards in the set Coldsnap with the highest converted mana cost."	1 card	12 tied cards
"What are the expenses of the budget with the lowest remaining?"	1 expense	2 tied expenses
"How many of the banned cards are white border?"	258 rows	89 cards
"… normal glutamic pylvic transaminase level, how many of them are male?"	1,924 lab records	45 patients
"… diagnosed with SJS, how many of them have a normal level of total protein?"	780 lab records	37 patients

value, BIRD and Spider references return one row in over 90% of silent questions, but the Geo and Restaurants questions that Spider imported from older datasets return one row in 21 of 247. Asked for a count over a join, BIRD references count joined rows in most silent questions, and the imported Academic, IMDB and Yelp questions in none of 39. Each group keeps its convention in training and development data alike, so examples retrieved from a benchmark's training data can carry its authors' convention into a model's answers. For the same wording, a model that follows BIRD's convention is therefore scored right on BIRD and wrong on the imported Spider questions. The score measures agreement with the authors, not correctness.

6.2 The unstated reading changes answers

The choice a reference makes silently is not cosmetic. For each BIRD development question that is silent on a choice, we apply the library's edit for the other reading to the reference SQL wherever it gives a valid reading, and run both. Of 664 such questions, 156 (23%) change their answer (Figure 16). Counting matters most, often by more than half of the value. For "How many accounts in North Bohemia has made a transaction with the partner's bank

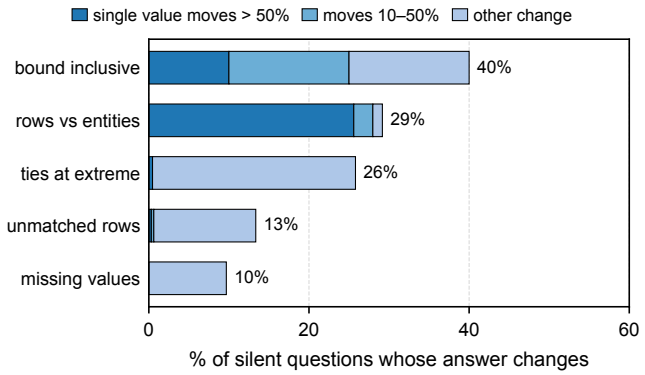


Figure 16: Silent BIRD development questions whose answer changes under the other reading, by choice type and by how far a single value moves.

being AB?", the reference counts joined rows and returns 2,260, while counting each account once returns 55. Ties change about a quarter of answers, often the list of returned items, and join type and missing-value filters change about one in ten. The data behind these choices are common. Every one of the 12 evaluation databases has at least two kinds of data on which an unstated choice changes an answer, such as joins that repeat rows, columns with missing values and maxima shared by several rows.

BIRD scores the other reading as wrong. A model that takes the unstated reading of a silent question loses accuracy for an answer the question allows. We ask the four models 137 BIRD development questions on which a tie, counting or join edit of the reference changes the answer, and score each answer against both queries. An author reviewed every question on which an answer matched only the edited query; the edits rejected removed DISTINCT, edited one side of a ratio, or turned a join that acts as a condition into an optional one, and a rule that excludes these three reproduces all 39 verdicts. Of the 233 answers BIRD marks wrong on the 79 questions the rule keeps, 102 return the other reading. On tie and counting questions, more answers match the edited reference (94) than the reference itself (37), and on 18 questions every model returns the other reading (Table 4). Accepting both readings raises accuracy on these questions from 26% to 59%.

7 TWO DESIGN RULES FOR RELIABLE ANSWERS

Models usually compute a missed reading once it is asked for on its own (Section 5). A system can therefore ask for each reading separately and let execution decide which readings differ. This section tests two rules built on this observation. The first shows an alternative only when its result differs (Sections 7.1–7.2), and the second carries a clarification by adding one clause to the request rather than rewriting it (Section 7.3).

7.1 Method

The method asks once for the readings of a request and answers each in its own call (Figure 17). One call lists the readings, each rewritten as a request with exactly one reading. If there is only

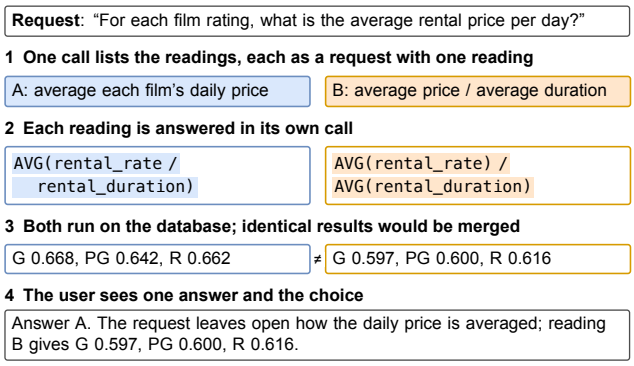


Figure 17: The disclosure method on the film request (an illustration). Blue and orange: readings A and B, as in Figure 1.

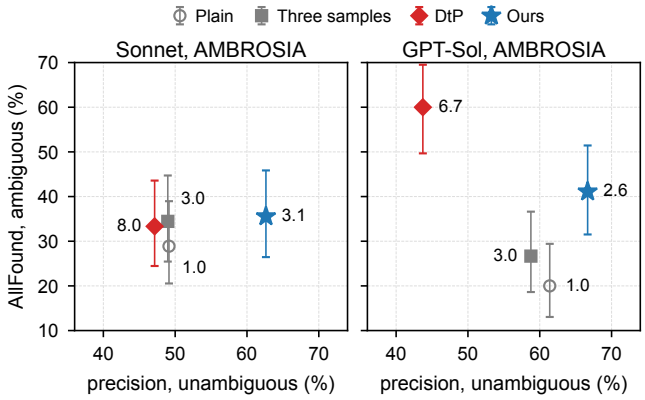


Figure 18: Disclosure methods on 150 new AMBROSIA test questions per model, with calls per question beside each point (Wilson 95% intervals).

one, one more call answers it and nothing else runs. Otherwise all answers are executed, readings with identical results are merged, and the first is returned with the others as alternatives. Whether two readings differ depends on the data, so execution can show that readings differ but not that they are equivalent [36], and a merged choice may matter on other data.

7.2 Show an alternative only when its result differs

We developed the method on 60 AMBROSIA questions and then froze it. A variant that lists the readings twice was dropped by a rule fixed in advance, because it lowered precision by more than two points. Two criteria were also fixed before the final test, namely higher precision on unambiguous questions than DtP, and AllFound within ten points of it. The final test uses 150 new test questions per model (90 ambiguous, 60 unambiguous) and compares four methods, run interleaved with the same model: the authors' prompt (plain); DtP with its released prompts, its fine-tuned infilling step replaced by that step's prompt; the authors' prompt sampled three times, at about our cost (three samples); and our method.

Table 5: Disclosure methods on the AMBROSIA final test and on 111 request sets. Bold: best per model.

		AMBROSIA			Request sets	
Model	Method	Precision (%)	All found (of 90)	Calls per q.	Both (of 111)	Specific right
Sonnet	Plain	49.1	26	**1.0**	64	**109**
	Three samples	49.0	31	3.0	72	108
	DtP	47.1	30	8.0	10	72
	Ours	**62.6**	**32**	3.1	**78**	**109**
GPT-Sol	Plain	61.4	18	**1.0**	47	92
	Three samples	58.8	24	3.0	60	92
	DtP	43.7	**54**	6.7	35	84
	Ours	**66.7**	37	2.6	**62**	**106**

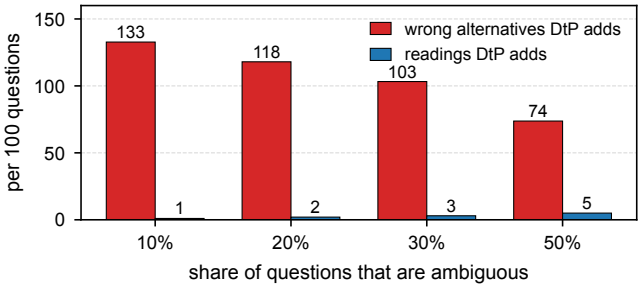


Figure 19: What DtP adds over our method per 100 questions, both models pooled: many wrong alternatives on clear questions for few extra readings.

Our method is the most precise (Figure 18). Pooled over both models, it is 19.2 points more precise than DtP ([12.3, 26.1]) at about a third of DtP's calls, because it shows an alternative only when its result differs. DtP is even less precise than plain, because it adds readings to clear questions. The coverage criterion fails, because DtP returns every reading for more ambiguous questions, by 8.3 points pooled, mainly with GPT-Sol, which lists more readings under DtP.

Precision matters because most requests are clear. A disclosure step runs on every request, and most requests do not leave a choice open; only 2 of the held-out BIRD failures are open choices. False alarms cost trust, since developers ignore tools whose false positives waste their time [21]. With GPT-Sol, if a fifth of the questions are ambiguous, DtP shows about 25 wrong alternatives for each extra reading it finds. Pooled over both models, DtP breaks even only if 94% of questions are ambiguous (Figure 19).

On request sets, the method also finds more. On 111 request sets (Table 5), the order reverses, and our method returns both readings for far more open requests than DtP, by 61 points for Sonnet and 24 for GPT-Sol, and answers more specific requests correctly. Against three samples at the same cost, the lead is small and not significant, 5 points for Sonnet and 2 for GPT-Sol, so on request sets the method's advantage lies over DtP. DtP names the choice most often, but its short restatements lose other requirements, which we test next.

Request (financial database); shaded: the requirements the rewrite lost

"For each district, list its id, its region, the client count, that count divided by its region's total, and that count divided by the overall total. Show the 10 districts …"

Rewritten as one interpretation

"Clients are attributed to the district of each account they are linked to via disp…"
SQL from the rewrite: wrong, without the shaded requirements

Kept, with one clause added for the reading

the request above, unchanged, + one sentence naming the reading
SQL: right

Figure 20: What a rewrite loses: one request set reading on the financial database.

7.3 Edit the request, do not rewrite it

Many systems rewrite a request before the SQL step. Retrieval chat frameworks such as LangChain's ConversationalRetrievalChain [8] and LlamaIndex's CondenseQuestionChatEngine [15] condense each follow-up into a standalone question. Snowflake's text-to-SQL service, Cortex Analyst, rewrites each question into a self-contained one before it writes SQL [32], conversational text-to-SQL parsers are trained to make the same rewrite [2, 31], and DtP writes each reading as a short interpretation. Each rewrite is a new copy that can lose a requirement. Figure 20 shows a rewrite on the financial database that dropped the region, the region share and the top-ten cut, while the request with one added clause was answered right.

Our request sets follow the opposite rule, since the three requests differ only in the choice, so we test which rule a clarification should follow. For each reading of a request set, the same SQL prompt receives one of four texts: (a) the original specific request; (b) the open request rewritten by the model as one interpretation with DtP's prompt, told the intended reading; (c) the open request unchanged, followed by a one-sentence description of the reading; (d) the open request condensed by the model into a standalone question with LangChain's condense-question prompt [8], with the open request as the earlier turn and the reading as the follow-up. Texts (b) to (d) start from the same open request and the same description. We report the 363 readings from accepted sets.

Keeping the request and adding one clause computes 85.4% of the readings correctly, as many as the original specific requests (Figure 21). Both rewrites compute fewer, 6.9 points fewer for the interpretation rewrite and 11.6 for the standalone question, and the loss is largest for Haiku and GPT-Luna. The rewrite seldom loses the choice itself. Of the readings answered correctly from the original request and wrongly from the rewrite, the judge finds the choice lost in only 6%; the rest lose another requirement, most often an output column, a formula or a join (Figure 21, right). On `insider`, Sonnet's rewrite turned "its average minus the baseline" into "its deviation from it", and the SQL took the absolute deviation; on Formula 1, Haiku's rewrite dropped "one row", and the SQL returned one row per race name instead of one row.

The same holds on real questions. We repeat the comparison on 85 BIRD development questions where one library edit of the reference SQL changes the answer. A simulated user follows each question with one sentence that settles the choice, such as "If several

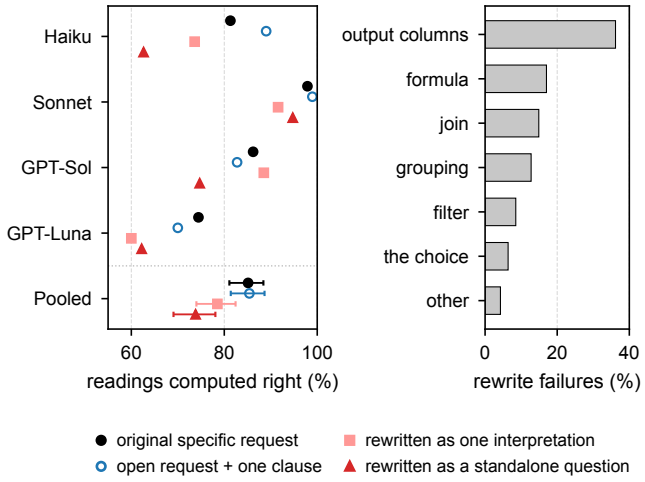


Figure 21: Editing versus rewriting a request (363 readings). Left: readings computed right from four request texts. Right: what the rewrite lost.

rows tie at the cut-off, return all of them," and the edited reference is the gold answer. Over 170 question–model pairs with Sonnet and GPT-Sol, adding the clause beats the standalone question, 58.2% against 47.6%. As on our request sets, the rewrite keeps the clarified choice and loses something else; the gap is largest for ties. Our method's first call also rewrites the request per reading and can inherit this loss, as can retrieval chat, planners and hand-offs between agents.

8 IMPLICATIONS

Systems could check readings on the user's data. Our results suggest that a model states its reading accurately and can usually compute the other one, yet seldom reports it. A system may therefore not need a stronger model to report open choices; a step that asks for each reading, executes the answers, and shows an alternative only when the results differ can already help. The data, not the wording alone, decide whether a choice matters. The two readings of the film request differ on `sakila_1` for every rating, but on a table where every film rents for the same number of days they return the same answer. When the user then settles the choice, keeping the original request and adding one clause appears safer than a rewrite, which can drop a column, a formula or a join that the user asked for.

Reporting could be a training target. The models we test already compute both readings of most requests; what they mostly lack is the habit of reporting the second one. This points to training rather than scale as a promising direction, since Sonnet, the strongest model, still misses the other reading in four of ten sets where it can compute it. Request sets could supply the signal such training needs. Each reading has its own executable reference, so a reward can credit an open answer only when it returns both readings and charge a specific answer for every alternative it adds.

Benchmarks could reward stating assumptions. Current benchmarks score a single answer per question, which suits questions with one correct reading. For a question that is silent on a

choice, a complementary practice is to let the model state the assumption it made and the alternative it set aside, and to credit both when each matches its own reference. Request sets make this possible because each reading has its own reference, so a benchmark can report accuracy on specific requests, disclosure on open requests, and restraint on settled ones. The library's SQL edit produces the second reference query from the first, as Section 6.1 does for BIRD, and could be accepted as a second answer wherever a question is silent on the choice, beginning with counting units and ties. Request sets can also be built on a benchmark's own databases at little cost. A new database needs one model review of its column meanings, and a new choice type one SQL edit that then applies to every database.

9 RELATED WORK

Natural language interfaces to databases. NaLIR [13] fixed "easy" ambiguities quietly and, for each "hard" one, explained its interpretation and offered several for the user to choose from. Its ambiguities lie mainly in mapping words to schema elements and in how phrases attach. Our results show that the alternatives NaLIR asked users to choose among are now cheap to compute; what is missing is the report. Our method applies NaLIR's interaction to calculation choices and lets execution decide which choices are "hard". Only readings with different results are shown, and a choice that changes nothing stays hidden.

Evaluation. Spider tests generalization across schemas [34]; BIRD adds large databases and external knowledge [14]; KaggleDBQA uses realistic databases and documentation [9]; Spider 2.0 tests enterprise workflows [11]. SParC and CoSQL add context and dialogue [33, 35], and BIRD-INTERACT tests clarification and recovery [7]. It scores whether a system completes a task after asking the user, whereas request sets score whether it reports a choice it made without asking. TrustSQL rewards a model for abstaining when it cannot answer reliably and penalizes wrong answers [10]; request sets reward a different behaviour, answering while reporting the open choice. Test suites [36] showed that execution on one database accepts wrong queries whose results match by chance, and distill databases that separate them from the gold query. Our concern is the opposite error, in which a correct query for another valid reading is rejected because the reference fixes one reading. Pourreza and Rafiei re-evaluated top models on Spider and BIRD by hand and found a perfect score out of reach "due to the multiple interpretations that can be derived from the provided samples" [20], and Wretblad et al. found noise in BIRD's questions and references whose correction changed which method ranks first [30]. We measure the first problem by construction rather than inspection and keep possible reference errors apart from open choices, because a wrong reference and a valid but different one need different fixes.

Ambiguous requests. The closest benchmarks are AmbiQT [1] and AMBROSIA [22] (Table 1). AmbiQT makes Spider questions ambiguous by changing the schema, with synonym columns or tables, split tables and precomputed aggregates, and checks whether both gold queries appear in the top five outputs. AMBROSIA has human-written questions with scope, attachment and vagueness ambiguity on generated databases. Both locate ambiguity in the schema or the wording. The choices behind the benchmark conventions of Section 6.1, such as ties, counting units, missing values and aggregation level, lie in the calculation and fall outside both taxonomies. For code generation, Menet et al. [16] bound what execution feedback can achieve by a term set by the ambiguity of the task description; request sets measure this term in SQL.

Generation, SQL methods and semantic layers. DBPal [29] and GAZP [37] generate training pairs for a target database; DBPal fills seed templates from the schema and adds automatic paraphrases. We also start from the database, but for evaluation. A request set fixes the database and every requirement but one, and execution checks that the change alters the answer. RAT-SQL, PICARD, DIN-SQL, DAIL-SQL, CHASE-SQL and Alpha-SQL [4, 12, 18, 19, 24, 28] improve how the SQL is generated. Methods that retrieve training examples, such as DAIL-SQL, can carry the benchmark authors' conventions into the answers, so part of a benchmark gain may come from matching them. LookML has developers write each measure "once, in one place" [5], the dbt Semantic Layer centralizes metric definitions [3], and Cortex Analyst generates SQL from them [27]. A layer settles the choices its authors foresaw, usually a measure's formula and counting unit; per-request choices, such as ties at a cut-off or the rows a ratio includes, remain open.

10 CONCLUSION

Text-to-SQL models make choices without saying so. Request sets, which change only one calculation choice, make this visible. Four models compute both readings of most sets when each is asked for, yet return both for the open request in only 16–58% of those sets, and when they settle a choice silently, they take the reading that benchmark references take by convention. Benchmarks cannot see the gap, because their references take one reading by conventions that differ by author group; over two in five answers BIRD marks wrong on such questions give the other reading. Two design rules make answers more reliable, showing an alternative only when its result differs and clarifying a request by adding one clause. The models already hold the other reading; what is missing is a step that shows it, and a benchmark that asks for it.

Limitations and future work. The generator covers a subset of analytical SQL, and the library comes from two benchmarks on SQLite, so other benchmarks may hold choice types we have not seen. The request sets cover every choice type on which real references differ, except quantifier, which the generator does not yet build, and referent column, which code cannot read off a reference; for the latter, AMBROSIA's vagueness questions show the same gap. Models write the request sets and judge the explanations, but an author reviewed every set, and execution decides every accuracy and coverage number. One author accepted the sets, without model results; the conventions are read by rules that see one SQL form each, so their counts are lower bounds. The BIRD rescoring uses one call per question without exploration, so its shares may shift with stronger prompting. We measure what a system shows, not how analysts decide with it. A system that can ask the user, as in BIRD-INTERACT, may settle a choice before it answers; request sets can measure whether it asks only when a choice is open. Other SQL dialects, multi-turn sessions and a user study of how analysts act on a reported choice are the next settings to test.

REFERENCES

[1] Adithya Bhaskar, Tushar Tomar, Ashutosh Sathe, and Sunita Sarawagi. 2023. Benchmarking and Improving Text-to-SQL Generation under Ambiguity. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Singapore, 7053–7074. https://doi.org/10.18653/v1/2023.emnlp-main.436

[2] Linzheng Chai, Dongling Xiao, Zhao Yan, Jian Yang, Liqun Yang, Qian-Wen Zhang, Yunbo Cao, and Zhoujun Li. 2023. QURG: Question Rewriting Guided Context-Dependent Text-to-SQL Semantic Parsing. In *PRICAI 2023: Trends in Artificial Intelligence (Lecture Notes in Computer Science)*, Fenrong Liu, Arun Anand Sadanandan, Duc Nghia Pham, Petrus Mursanto, and Dickson Lukose (Eds.). Springer Nature Singapore, Singapore, 275–286. https://doi.org/10.1007/978-981-99-7022-3_24

[3] dbt Labs. 2025. dbt Semantic Layer. dbt documentation. https://docs.getdbt.com/docs/use-dbt-semantic-layer/dbt-sl

[4] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. 2024. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proceedings of the VLDB Endowment* 17, 5 (2024), 1132–1145. https://doi.org/10.14778/3641204.3641221

[5] Google Cloud. 2025. Introduction to LookML. Looker documentation. https://docs.cloud.google.com/looker/docs/what-is-lookml

[6] Thomas Herndon, Michael Ash, and Robert Pollin. 2014. Does High Public Debt Consistently Stifle Economic Growth? A Critique of Reinhart and Rogoff. *Cambridge Journal of Economics* 38, 2 (2014), 257–279.

[7] Nan Huo, Xiaohan Xu, Jinyang Li, Per Jacobsson, Shipei Lin, Bowen Qin, Binyuan Hui, Xiaolong Li, Ge Qu, Shuzheng Si, Linheng Han, Edward Alexander, Xintong Zhu, Rui Qin, Ruihan Yu, Yiyao Jin, Feige Zhou, Weihao Zhong, Yun Chen, Hongyu Liu, Chenhao Ma, Fatma Ozcan, Yannis Papakonstantinou, and Reynold Cheng. 2026. BIRD-INTERACT: Re-imagining Text-to-SQL Evaluation via Lens of Dynamic Interactions. In *International Conference on Learning Representations*, C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust (Eds.), Vol. 2026. ICLR, Rio de Janeiro, Brazil, 44285–44333. https://proceedings.iclr.cc/paper_files/paper/2026/hash/496b549556509bbb9770bf9d335c5800-Abstract-Conference.html

[8] LangChain. 2024. ConversationalRetrievalChain and its condense-question prompt (CONDENSE_QUESTION_PROMPT). LangChain documentation and source, `langchain/chains/conversational_retrieval`. https://github.com/langchain-ai/langchain/blob/v0.3/libs/langchain/langchain/chains/conversational_retrieval/prompts.py

[9] Chia-Hsuan Lee, Oleksandr Polozov, and Matthew Richardson. 2021. KaggleDBQA: Realistic Evaluation of Text-to-SQL Parsers. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Association for Computational Linguistics, Online, 2261–2273. https://doi.org/10.18653/v1/2021.acl-long.176

[10] Gyubok Lee, Woosog Chay, Seonhee Cho, and Edward Choi. 2024. TrustSQL: Benchmarking Text-to-SQL Reliability with Penalty-Based Scoring. arXiv:2403.15879 https://arxiv.org/abs/2403.15879

[11] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin SU, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida Wang, and Tao Yu. 2025. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. In *International Conference on Learning Representations*, Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (Eds.), Vol. 2025. ICLR, Singapore, 28691–28735. https://proceedings.iclr.cc/paper_files/paper/2025/hash/46c10f6c8ea5aa6f267bcdabcb123f97-Abstract-Conference.html

[12] Boyan Li, Jiayi Zhang, Ju Fan, Yanwei Xu, Chong Chen, Nan Tang, and Yuyu Luo. 2025. Alpha-SQL: Zero-Shot Text-to-SQL using Monte Carlo Tree Search. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research)*, Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (Eds.), Vol. 267. PMLR, Vancouver, Canada, 36810–36830. https://proceedings.mlr.press/v267/li25dt.html

[13] Fei Li and H. V. Jagadish. 2014. Constructing an Interactive Natural Language Interface for Relational Databases. *Proceedings of the VLDB Endowment* 8, 1 (2014), 73–84. https://www.vldb.org/pvldb/vol8/p73-li.pdf

[14] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., Red Hook, NY, USA, 42330–42357. https://doi.org/10.52202/075280-1835

[15] LlamaIndex. 2024. Chat Engine – Condense Question Mode (CondenseQuestionChatEngine). LlamaIndex documentation. https://developers.llamaindex.ai/python/examples/chat_engine/chat_engine_condense_question/

[16] Nicolas Menet, Michael Hersche, Andreas Krause, and Abbas Rahimi. 2026. A Theoretical Analysis of Test-Driven Code Generation. arXiv:2602.06098 https://arxiv.org/abs/2602.06098

[17] Ziad Obermeyer, Brian Powers, Christine Vogeli, and Sendhil Mullainathan. 2019. Dissecting Racial Bias in an Algorithm Used to Manage the Health of Populations. *Science* 366, 6464 (2019), 447–453.

[18] Mohammadreza Pourreza, Hailong Li, Ruoxi Sun, Yeounoh Chung, Shayan Talaei, Gaurav Tarlok Kakkar, Yu Gan, Amin Saberi, Fatma Ozcan, and Sercan Arik. 2025. CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL. In *International Conference on Learning Representations*, Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (Eds.), Vol. 2025. ICLR, Singapore, 60385–60415. https://proceedings.iclr.cc/paper_files/paper/2025/hash/974ff7b5bf08dbf9400b5d599a39c77f-Abstract-Conference.html

[19] Mohammadreza Pourreza and Davood Rafiei. 2023. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. In *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., Red Hook, NY, USA, 36339–36348. https://doi.org/10.52202/075280-1577

[20] Mohammadreza Pourreza and Davood Rafiei. 2023. Evaluating Cross-Domain Text-to-SQL Models and Benchmarks. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 1601–1611. https://doi.org/10.18653/v1/2023.emnlp-main.99

[21] Caitlin Sadowski, Edward Aftandilian, Alex Eagle, Liam Miller-Cushon, and Ciera Jaspan. 2018. Lessons from Building Static Analysis Tools at Google. *Commun. ACM* 61, 4 (2018), 58–66.

[22] Irina Saparina and Mirella Lapata. 2024. AMBROSIA: A Benchmark for Parsing Ambiguous Questions into Database Queries. In *Advances in Neural Information Processing Systems*, A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang (Eds.), Vol. 37. Curran Associates, Inc., Red Hook, NY, USA, 90600–90628. https://doi.org/10.52202/079017-2876

[23] Irina Saparina and Mirella Lapata. 2025. Disambiguate First, Parse Later: Generating Interpretations for Ambiguity Resolution in Semantic Parsing. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 16825–16839. https://doi.org/10.18653/v1/2025.findings-acl.863

[24] Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 9895–9901. https://doi.org/10.18653/v1/2021.emnlp-main.779

[25] David M. Shahian, Robert E. Wolf, Lisa I. Iezzoni, Leslie Kirle, and Sharon-Lise T. Normand. 2010. Variability in the Measurement of Hospital-Wide Mortality Rates. *New England Journal of Medicine* 363, 26 (2010), 2530–2539.

[26] Raphael Silberzahn, Eric L. Uhlmann, Daniel P. Martin, et al. 2018. Many Analysts, One Data Set: Making Transparent How Variations in Analytic Choices Affect Results. *Advances in Methods and Practices in Psychological Science* 1, 3 (2018), 337–356.

[27] Snowflake. 2025. Cortex Analyst. Snowflake documentation. https://docs.snowflake.com/user-guide/snowflake-cortex/cortex-analyst

[28] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. 2020. RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Online, 7567–7578. https://doi.org/10.18653/v1/2020.acl-main.677

[29] Nathaniel Weir, Prasetya Utama, Alex Galakatos, Andrew Crotty, Amir Ilkhechi, Shekar Ramaswamy, Rohin Bhushan, Nadja Geisler, Benjamin Hättasch, Steffen Eger, Ugur Cetintemel, and Carsten Binnig. 2020. DBPal: A Fully Pluggable NL2SQL Training Pipeline. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, New York, NY, USA, 2347–2361. https://doi.org/10.1145/3318464.3380589

[30] Niklas Wretblad, Fredrik Riseby, Rahul Biswas, Amin Ahmadi, and Oskar Holmström. 2024. Understanding the Effects of Noise in Text-to-SQL: An Examination of the BIRD-Bench Benchmark. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 356–369. https://doi.org/10.18653/v1/2024.acl-short.34

[31] Dongling Xiao, Linzheng Chai, Qian-Wen Zhang, Zhao Yan, Zhoujun Li, and Yunbo Cao. 2022. CQR-SQL: Conversational Question Reformulation Enhanced Context-Dependent Text-to-SQL Parsers. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.). Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 2055–2068. https://doi.org/10.18653/v1/2022.findings-emnlp.150

[32] Aiwen Xu. 2024. Snowflake Cortex Analyst: Support for Multi-turn Conversation. Snowflake Engineering Blog. https://www.snowflake.com/en/engineering-blog/

cortex-analyst-multi-turn-conversations-support/

[33] Tao Yu, Rui Zhang, Heyang Er, Suyi Li, Eric Xue, Bo Pang, Xi Victoria Lin, Yi Chern Tan, Tianze Shi, Zihan Li, Youxuan Jiang, Michihiro Yasunaga, Sungrok Shim, Tao Chen, Alexander Fabbri, Zifan Li, Luyao Chen, Yuwen Zhang, Shreya Dixit, Vincent Zhang, Caiming Xiong, Richard Socher, Walter Lasecki, and Dragomir Radev. 2019. CoSQL: A Conversational Text-to-SQL Challenge Towards Cross-Domain Natural Language Interfaces to Databases. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, Hong Kong, China, 1962–1979. https://doi.org/10.18653/v1/D19-1204

[34] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. 2018. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Brussels, Belgium, 3911–3921. https://doi.org/10.18653/v1/D18-1425

[35] Tao Yu, Rui Zhang, Michihiro Yasunaga, Yi Chern Tan, Xi Victoria Lin, Suyi Li, Heyang Er, Irene Li, Bo Pang, Tao Chen, Emily Ji, Shreya Dixit, David Proctor, Sungrok Shim, Jonathan Kraft, Vincent Zhang, Caiming Xiong, Richard Socher, and Dragomir Radev. 2019. SParC: Cross-Domain Semantic Parsing in Context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, Florence, Italy, 4511–4523. https://doi.org/10.18653/v1/P19-1443

[36] Ruiqi Zhong, Tao Yu, and Dan Klein. 2020. Semantic Evaluation for Text-to-SQL with Distilled Test Suites. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Online, 396–411. https://doi.org/10.18653/v1/2020.emnlp-main.29

[37] Victor Zhong, Mike Lewis, Sida I. Wang, and Luke Zettlemoyer. 2020. Grounded Adaptation for Zero-shot Executable Semantic Parsing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Online, 6869–6882. https://doi.org/10.18653/v1/2020.emnlp-main.558